

P1640R1

Error size benchmarking: Redux

Published Proposal, 2019-09-29

This version:

<https://wg21.link/P1640R1>

Author:

[Ben Craig](#) (National Instruments)

Audience:

WG21; DG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/ben-craig/error_bench/blob/master/bench_blog.bs

Abstract

The author measures size costs of error handling approaches. Exceptions are big, `std::abort` and integer error codes are small.

Table of Contents

- 1 Introduction**
- 2 Changes**
- 3 Measuring methodology**
- 4 Starter test cases**
- 5 Measurements**
 - 5.1 Initial error neutral size cost
 - 5.2 Incremental error neutral size cost

- 5.3 Initial size cost of signaling an error
- 5.4 Incremental size cost of signaling an error
- 5.5 Initial size cost for handling an error
- 5.6 Incremental size cost for handling an error

6 Conclusion

7 Acknowledgments

Appendix A: Why no speed measurements?

Appendix B: The build flags

MSVC

Clang x64

GCC x64

Appendix C: The code

Common support code

Initial error neutral functions

Incremental error neutral functions

Initial signaling of an error

Incremental signaling of an error

Initial handling of an error

Incremental handling of an error

Appendix D: Linear graphs

Initial error neutral cost, linear

Initial cost of signaling an error, linear

References

Informative References

“

To make progress, we need better data on costs and performance to evaluate the -
often simplistic and narrowly focused - solutions suggested.

— *Direction for ISO C++* [\[P0939R2\]](#)

”

§ 1. Introduction

In this paper, we will look at the *size* costs of error handling. We'll break things down into one-time costs and incremental costs, and subdivide by costs paid for error neutral functions, raising an error, and handling an error.

§ 2. Changes

R1 of this paper removed the discussion on how exceptions are implemented, and reduced the set of error handling strategies down to throwing exceptions, return codes, abort, and noexcept abort. This is to make the paper and charts easier to read. The data is the same.

R1 renamed the "stripped" cases to "baremetal" cases, to better frame where those numbers would be applicable.

§ 3. Measuring methodology

All benchmarks lie. It's important to know how a benchmark is set up so that the useful parts can be distinguished from the misleading parts.

The specific build flags can be found in [Appendix B](#). Following is a brief summary.

MSVC 2019 was used for MSVC x86 and MSVC x64 builds. The /d2FH4 flag described in [\[MoFH4\]](#) was used, and /EHs was used when exceptions were on.

GCC 7.3.1 from the Red Hat Developer Toolset 7.1 was used for my GCC builds. The Linux x64 platform was targeted.

Clang 8.0.0, libc++, and libc++abi was used for my Clang builds. The Linux x64 platform was targeted. The system linker and C library leaked in to this build. The system GCC was GCC 4.8.4 from Ubuntu 14.04.3.

All the binaries are optimized for size, rather than speed.

All the binaries are built with static runtimes, so that we can also see the costs of the error handling runtime machinery. For many people, this is a sunk cost. If the cost of the runtime machinery isn't of interest, then don't pay attention to the one-time costs, and just look at the incremental costs. Sizes were not calculated by just doing the "easy" thing and comparing the on-disk sizes of the resulting programs. Programs have lots and lots of padding internal to them due to alignment constraints, and that padding can mask or inflate small cost changes. Instead, the size is calculated by summing the size of all the non-code sections, and by summing the size of each function in the code sections. Measuring the size of a function is

a little tricky, as the compiler doesn't emit that information directly. There are often padding instructions between consecutive functions. My measurements omit the padding instructions so that we can see code size differences as small as one byte.

Measurements are also included where the size of some data sections related to unwinding are omitted. On x64 Linux, programs can have an `.eh_frame` and `.eh_frame_hdr` section that can help with emitting back traces. x64 Windows has similar sections named `.xdata` and `.pdata`. These sections aren't sufficient to implement C++ exception handling, and they don't go away when exceptions are turned off. On Linux and Windows, these sections should be considered a sunk cost, but on more exotic platforms, it is reasonable to omit those sections, as stack trace costs may not be tolerable. These measurements are all labeled as "baremetal". x86 Windows doesn't have these sections, so the "baremetal" measurements are the same as the regular measurements.

Note that on Linux, the entire user mode program can be statically linked. This is the program under test, the C++ runtime, the C runtime, and any OS support. On Windows, the program, the C++ runtime, and the C runtime can be statically linked, but the OS support (`kernel32.dll`) is still distinct. With this in mind, refrain from comparing the one-time MSVC sizes to the Clang and GCC sizes, as it isn't comparing the same set of functionality.

These benchmarks are run on very small programs. On larger programs, various code and data deduplication optimizations could substantially change the application-level costs of error handling. [\[MoFH4\]](#) documents the kinds of deduplication that MSVC 2019 performs.

§ 4. Starter test cases

To start with, we will look at code similar to the following:

```
struct Dtor {~Dtor() {}};
int global_int = 0;
void callee() {/* will raise an error one day*/}
void caller() {
    Dtor d;
    callee();
    global_int = 0;
}
int main() {
    caller();
    return global_int;
}
```

This code has some important properties for future comparisons.

- `callee()` will eventually raise errors.
- `caller()` needs to clean up the `d` object in error and non-error conditions.
- `caller()` should only set `global_int` in success cases.
- The code doesn't have any error cases yet. We can see the cost of error handling machinery when no errors are involved.

In the actual tests all the function bodies are in separate .cpp files, and link-time / whole-program optimizations aren't used. If they had been used, the entire program would get optimized away, removing our ability to measure error handling differences.

The above program is a useful template when using exceptions or `std::abort` as an error handling mechanism, but it won't work as well for error codes. So we mutate the program like so...

```
int callee() {return 0;}
int caller() {
    Dtor d;
    int e = callee();
    if (e)
        return e;
    global_int = 0;
    return e;
}
```

This is pretty typical integer return value code, without any macro niceties.

Most of the programs were built with exceptions turned off, but the `throw_*` cases and `noexcept_abort` all had exceptions turned on in the program.

- `abort`: When an error is encountered, kill the program with `std::abort`.
- `noexcept_abort`: Same as `abort`, except exceptions are turned on, and all the functions declared in user source are marked as `noexcept`.
- `return_val`: Return an integer error code, where zero represents success.
- `throw_exception`: Throw an exception deriving from `std::exception`, that contains only an int. This should represent more typical use cases.

Data on many more error handling strategies can be found in [R0](#) of this paper.

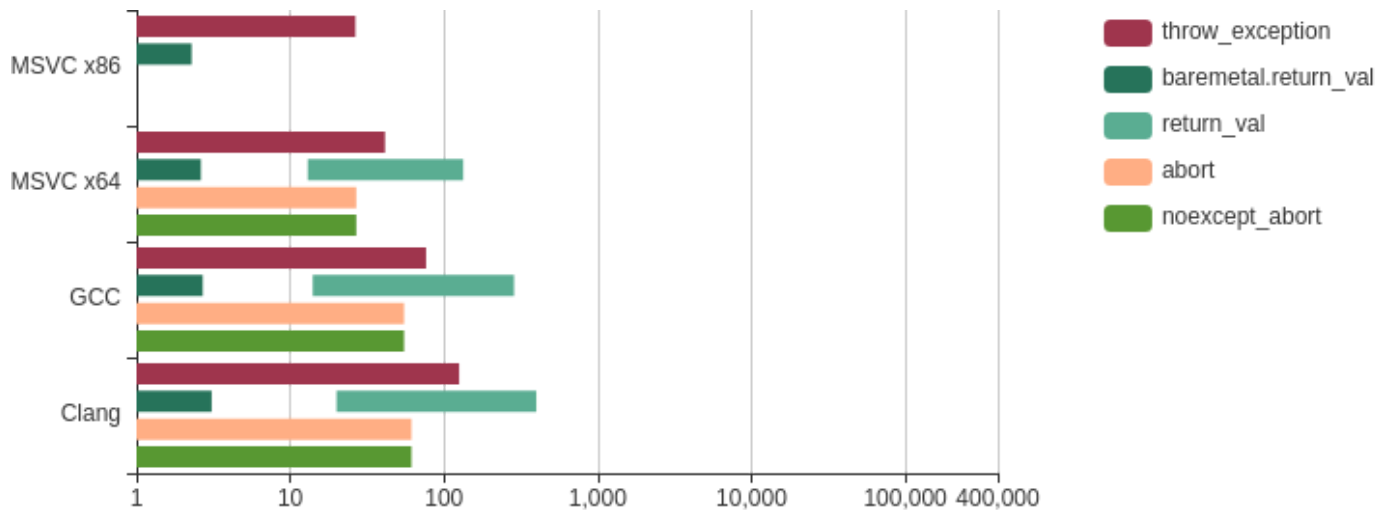
Expository code for all the cases can be found in [Appendix C](#). The actual code used for the benchmark can be found [on my github](#).

§ 5. Measurements

§ 5.1. Initial error neutral size cost

My first batch of measurements is comparing each of the mechanisms to the `baremetal.abort` test case. This lets us focus on the incremental costs of the other mechanisms.

Warning! Logarithmic axis! Linear version [here](#)



Warning! Logarithmic axis! Linear version [here](#)

These tables show us that the one-time cost for exceptions is really high (6KB on MSVC x86, 382KB on Clang x64), and the one time cost for unwind information is pretty high too (6KB on MSVC x64, 57KB on Clang). Note that `noexcept_abort` has the same cost as regular `abort` right now. If everything is `noexcept`, the exception machinery costs are not incurred.

§ 5.2. Incremental error neutral size cost

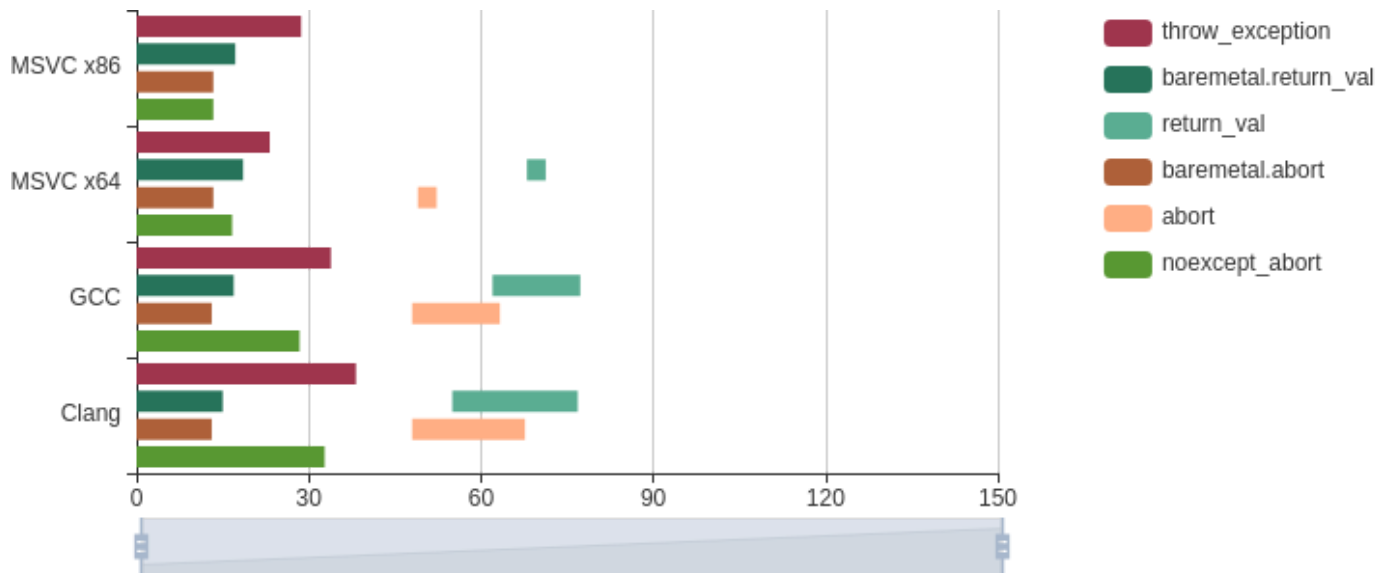
To measure the incremental cost of error neutral code, the code will be updated as follows:

```

void callee2(int amount) {
    global_int += amount;
    // will error one day
}
void caller2(int amount) {
    Dtor d;
    callee2(amount);
    global_int += amount;
}
int main() {
    caller();
    caller2(0);
    return global_int;
}

```

The "2" versions of these functions are slightly different than the original versions in order to avoid optimization where identical code is de-duplicated (COMDAT folding). Each error handling case was updated to the idiomatic form that had the same semantics as this error neutral form. Here are the incremental numbers:



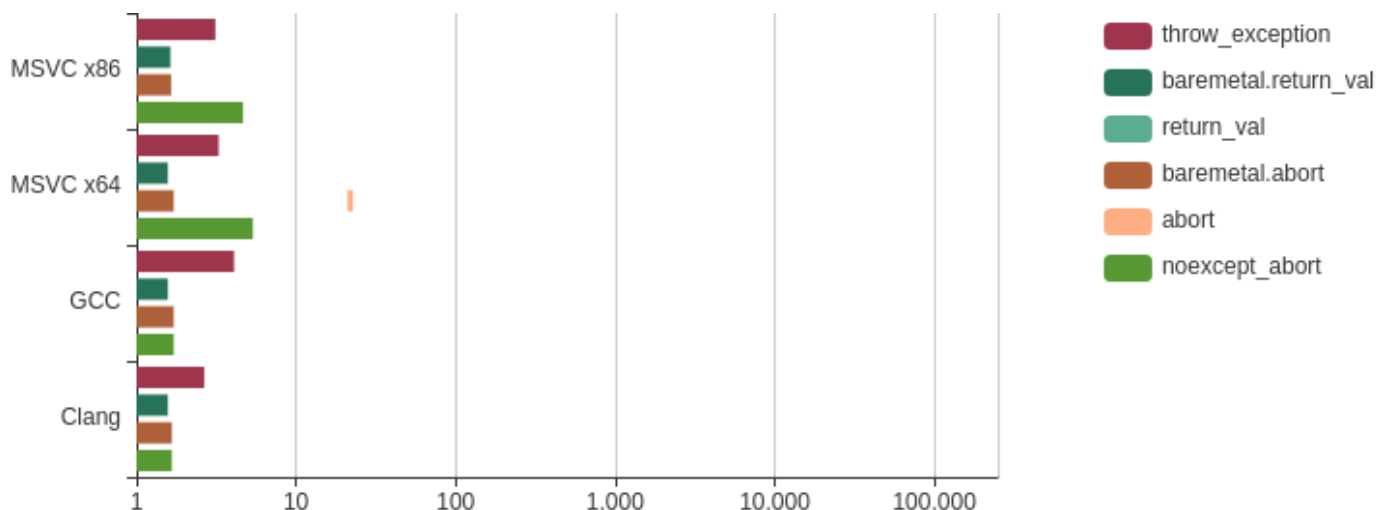
The delta between the best and the worst is much smaller in the incremental error neutral measurements than in the one-time cost measurements. `abort` and return values were always cheaper than exceptions, even with included unwind information.

§ 5.3. Initial size cost of signaling an error

What happens when an error is signaled first time? What's the one-time cost of that first error?

```
void callee() {
    if (global_int == INT_MAX)
        throw 1;
}
```

Warning! Logarithmic axis! Linear version [here](#)



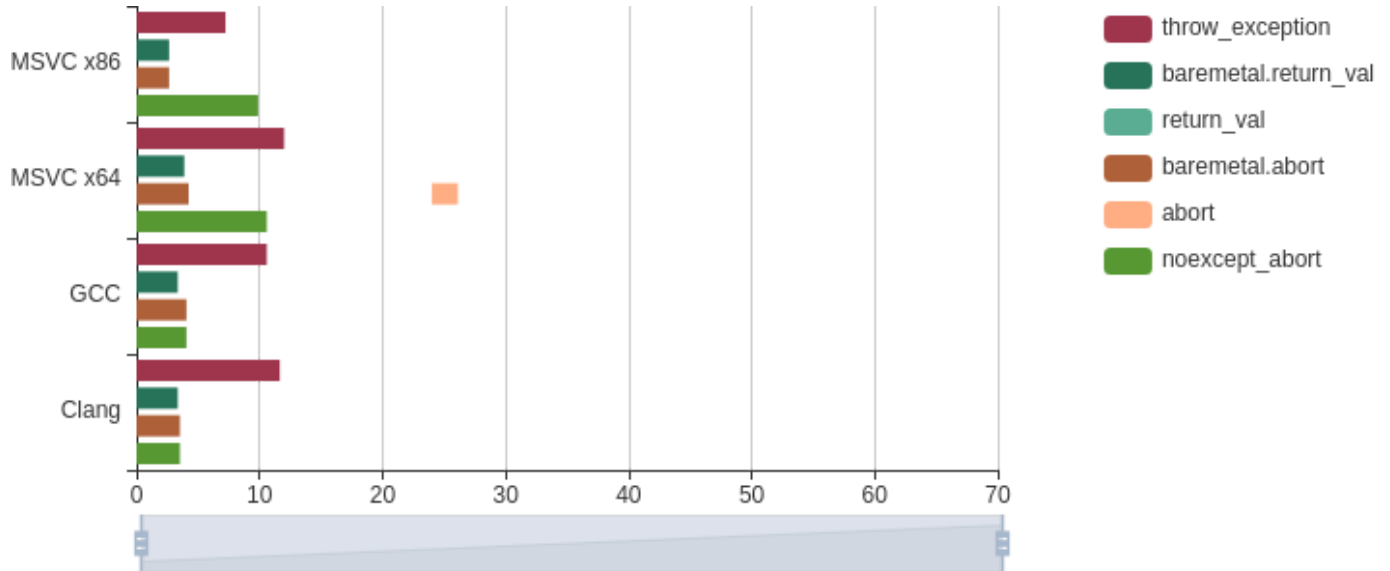
Warning! Logarithmic axis! Linear version [here](#)

On MSVC, there are multiple ways to build with exceptions "on". This experiment was built with `/EHs`, which turns on exceptions in a C++ conforming manner. The Microsoft recommended flag is `/EHsc`, which turns on exceptions for all C++ functions, but assumes that `extern "C"` functions won't throw. This is a useful, though non-conforming option. The trick is that the `noexcept_abort callee()` implementation calls `abort()`, and that's an `extern "C"` function that isn't marked as `noexcept`, so we suddenly need to pay for all the exception handling costs that we had been avoiding by making everything `noexcept`. We can't easily make the C runtime, or other people's code `noexcept`. We don't see this on GCC and Clang because the C library they are calling marks `abort` as `__attribute__((__nothrow__))`, and that lets them avoid generating the exception machinery.

GCC's first throw costs look worse than Clang's because Clang paid a lot of those costs even before there was a throw.

§ 5.4. Incremental size cost of signaling an error

```
void callee2(int amount) {  
    if (global_int + amount == INT_MAX)  
        throw 1;  
    global_int += amount;  
}
```



These numbers are all over the place. Here are some highlights:

- On GCC and Clang, throwing an exception is more incrementally expensive than all the non-throwing variants.
- On MSVC, it isn't just the first non-`noexcept` function that is expensive in `noexcept_abort`, but the later calls are expensive too.

§ 5.5. Initial size cost for handling an error

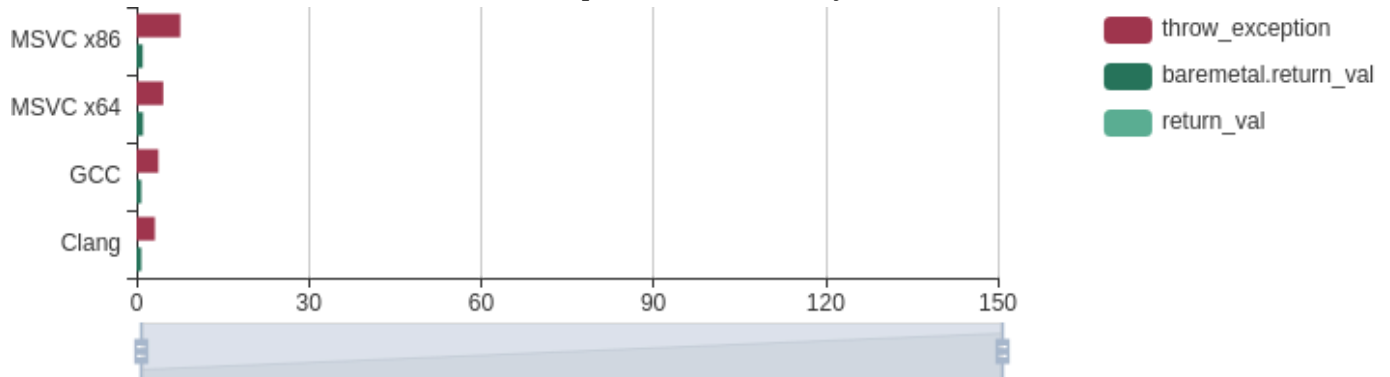
To get the initial handling costs, we'll rewrite main to look something like this...

```

int main() {
    try {
        caller();
    } catch (int) {
        global_int = 0;
    }
    caller2(0);
    return global_int;
}

```

`abort` results won't be included here, because there is no "handling" of an `abort` call in C++. The environment needs to handle it and restart the process, reboot the system, or relaunch the rocket.



Here we see that the initial catch cost exceptions is high compared to the alternatives.

§ 5.6. Incremental size cost for handling an error

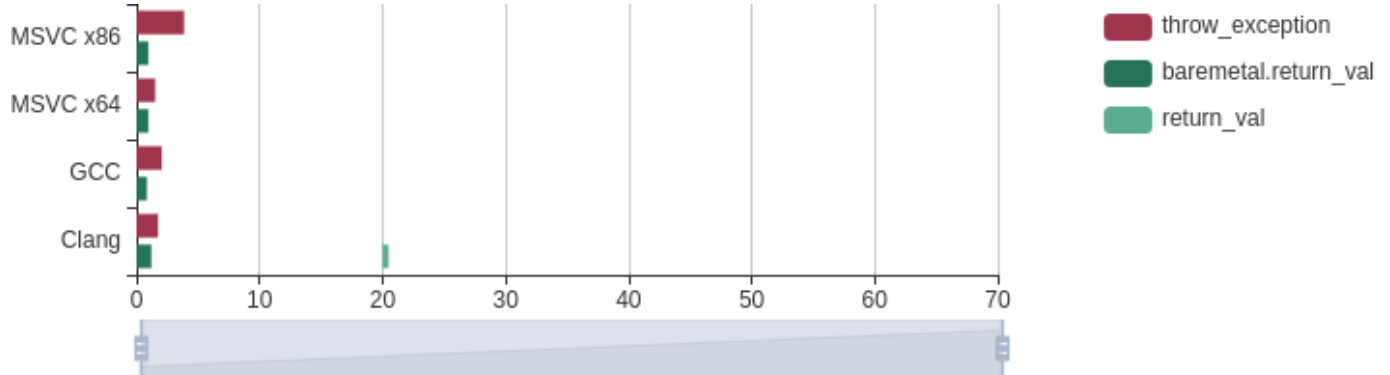
Now for the incremental code, and the associated costs.

```

int main() {
    try {
        caller();
    } catch (int) {
        global_int = 0;
    }
    try {
        caller2(0);
    } catch (int) {
        global_int = 0;
    }
    return global_int;
}

```

Note that this is measuring the cost of handling a second error within a single function. If the error handling were split over multiple functions, the cost profile may be different.



§ 6. Conclusion

Exceptions and on-by-default unwinding information are reasonable error handling strategies in many environments, but they don't serve all needs in all use cases. C++ needs standards conforming ways to avoid exception and unwind overhead on platforms that are size constrained. C++ is built on the foundation that you don't pay for what you don't use, and that you can't write the language abstractions better by hand. This paper provides evidence that you can write error handling code by hand that results in smaller code than the equivalent exception throwing code if all you use is terminate semantics or an integer's worth of error information. In each of the six test cases, terminate and integer return values beat exceptions on size, even before stripping out unwind information.

§ 7. Acknowledgments

Simon Brand, Niall Douglas, Brad Keryan, Reid Kleckner, Modi Mo, Herb Sutter, John McFarlane, Ben Saks, and Richard Smith provided valuable review commentary on R0 of this paper.

Thanks to Lawrence Crowl, for asking the question "what if everything were `noexcept`?".

Charts generated with [\[ECharts\]](#).

§ Appendix A: Why no speed measurements?

[\[P1886\]](#) contains speed measurements for various error handling strategies.

§ Appendix B: The build flags

§ MSVC

The compiler and flags are the same for 32-bit and 64-bit builds, except that the 32-bit linker uses /machine:x86 and the 64-bit linker uses /machine:x64

Compiler marketing version: Visual Studio 2019

Compiler toolkit version: 14.20.27508

cl.exe version: 19.20.27508.1

Compiler codegen flags (no exceptions): /GR /Gy /Gw /O1 /MT /d2FH4 /std:c++latest /permissive- /DNDEBUG

Compiler codegen flags (with exceptions): /EHs /GR /Gy /Gw /O1 /MT /d2FH4 /std:c++latest /permissive- /DNDEBUG

Linker flags: /OPT:REF /release /subsystem:CONSOLE /incremental:no /OPT:ICF /NXCOMPAT /DYNAMICBASE /DEBUG *.obj

§ Clang x64

Toolchains used:

- Clang 8.0.0 and libc++
- System linker from Ubuntu 14.04.3's GCC 4.8.4 installation

Compiler codegen flags (no exceptions): -fno-exceptions -Os -ffunction-sections -fdata-sections -std=c++17 -stdlib=libc++ -static -DNDEBUG

Compiler codegen flags (exceptions): -Os -ffunction-sections -fdata-sections -std=c++17 -stdlib=libc++ -static -DNDEBUG

Linking flags: -Wl,--gc-sections -pthread -static -static-libgcc -stdlib=libc++ *.o libc++abi.a

§ GCC x64

Toolchain used: GCC 7.3.1 from the Red Hat Developer Toolset 7.1

Compiler codegen flags (no exceptions): `-fno-exceptions -Os -ffunction-sections -fdata-sections -std=c++17 -static`

Compiler codegen flags (exceptions): `-Os -ffunction-sections -fdata-sections -std=c++17 -static`

Linking flags: `-Wl,--gc-sections -pthread -static -static-libgcc -static-libstdc++ *.o`

§ Appendix C: The code

As stated before, this isn't the exact code that was benchmarked. In the benchmarked code, functions were placed in distinct translation units in order to avoid inlining. The following code is provided to demonstrate what the error handling code looks like.

§ Common support code

► **Expand to see code snippets**

§ Initial error neutral functions

This section lays the groundwork for future comparisons. All of these cases are capable of transporting error information from a future signaling site (`callee`) to a future catching site (`main`). No errors are signaled here, but the plumbing is in place.

► **Expand to see code snippets**

§ Incremental error neutral functions

Here, we add an extra two functions with error transporting capabilities so that we can measure the incremental cost of error neutral functions. These functions need to be slightly different than the old functions in order to avoid deduplication optimizations.

In order to save on text length, the only functions that will be listed here are the functions were added or changed compared to the previous section.

► **Expand to see code snippets**

§ Initial signaling of an error

► **Expand to see code snippets**

§ Incremental signaling of an error

► Expand to see code snippets

§ Initial handling of an error

► Expand to see code snippets

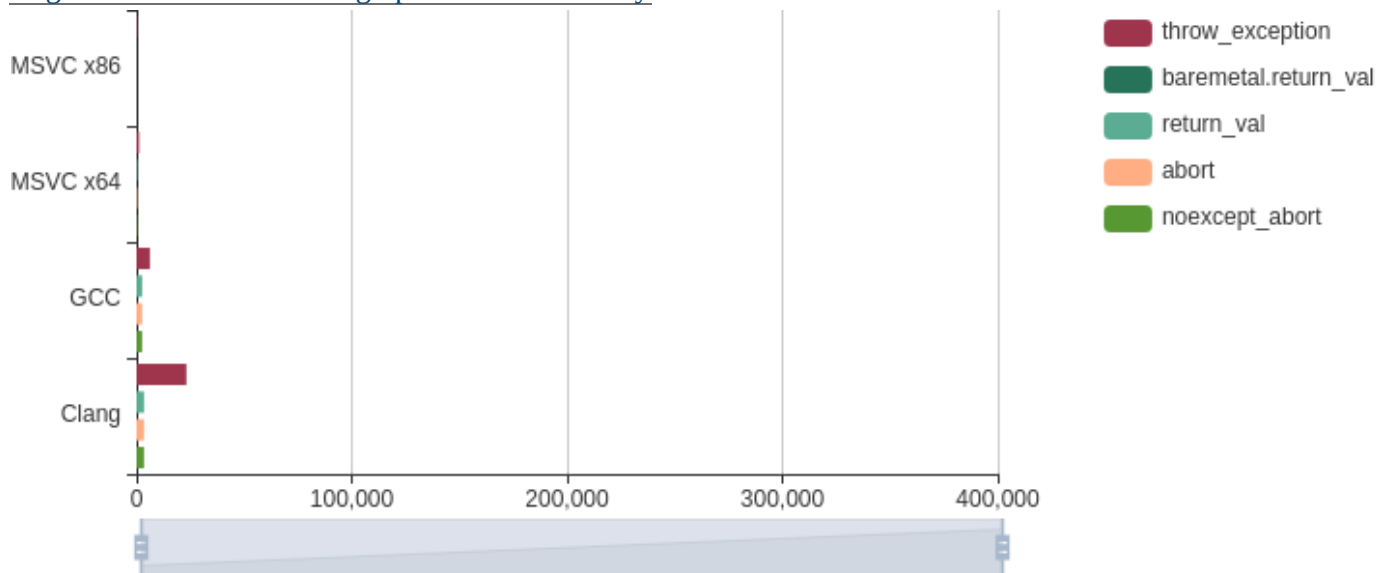
§ Incremental handling of an error

► Expand to see code snippets

§ Appendix D: Linear graphs

§ Initial error neutral cost, linear

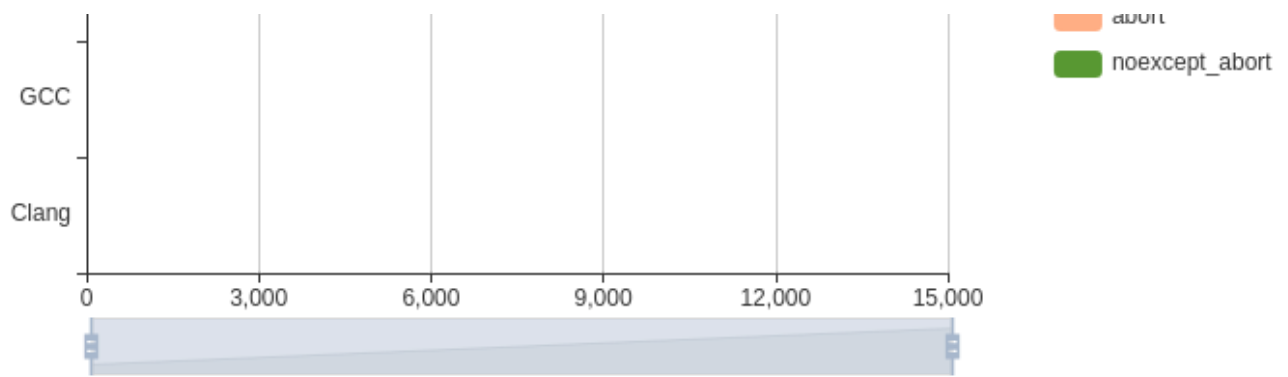
[Logarithmic version of this graph with commentary](#)



§ Initial cost of signaling an error, linear

[Logarithmic version of this graph with commentary](#)





§ References

§ Informative References

[ECharts]

ECharts. URL: <https://ecomfe.github.io/echarts-doc/public/en/index.html>

[MoFH4]

Modi Mo. Making C++ Exception Handling Smaller On x64. URL: <https://devblogs.microsoft.com/cppblog/making-cpp-exception-handling-smaller-x64/>

[P0939R2]

H. Hinnant; et al. Direction for ISO C++. URL: <http://wg21.link/P0939R2>

[P1886]

Ben Craig. Error speed benchmarking. URL: <http://wg21.link/P1866>